L E A L

CENTRO DE
INVESTIGACIÓN
DIGITAL

UNIVERSIDAD DE LA HABANA

LENGUAJE LEAL

BIBLIOTECA
Escuela de Matemática

LENGUAJE LEAL

DESARROLLADO EN EL
CENTRO DE INVESTIGACION DIGITAL

por:

Ing. Víctor M. Llopis

COLABORARON EN EL DESARROLLO DEL LEAL:

DISEÑO ORIGINAL Y ARITMETICA .	Lic. Víctor A. Orsini.
ENTRADA Y SALIDA	Cro. Emilio Lino.
SUBRUTINAS BASICAS	Cro. Fedor Sierra.
FUNCIONES TRASCENDENTES . . .	Cra. María E. Bragado. Cro. Miguel Fonfría.
REDACCION	Cro. Alejandro Dovale. Cra. Maida Maletá.

La Habana, Cuba
1971

ÍNDICE

I.—INTRODUCCION

1. General 7
2. Componentes de la CID 201-A 7
3. Lenguaje y Compilador 8

II.—DESCRIPCION DEL LEAL

1. Representación de los números en la Computadora 9
2. Estilo de Escritura 10
3. Operandos 11
4. Convenio adoptado en el texto 12
5. Subíndices o Sufijos 12
6. Paréntesis 13
7. Números de Referencia 13
8. Declaraciones 13
9. Instrucciones de entrada y salida 15
10. Instrucciones de Alto y Fin 16
11. Operadores y operaciones aritméticas 17
12. Funciones 19
13. Expresiones aritméticas 21
14. Instrucciones de Salto 22
15. Instrucciones de Ciclo 24
16. Subrutinas 31
17. Observaciones 37
18. Chequeo 37
19. Impresión 41
20. Error. 44
21. Rastreo o Chequeo 2 45
22. División de un programa 45

EJEMPLOS GENERALES 37

EJERCICIOS 40

I.—INTRODUCCION

1) General

La Cid-201-A es una Computadora electrónica de propósito general o sea que puede calcular programas que tengan distintas aplicaciones. Resuelve problemas mediante la ejecución de estos programas automáticamente y a gran velocidad. Esta máquina ha sido construida usando circuitos integrados por lo que es llamada de tercera generación. Su velocidad de operación es de 25,000 sumas / segundo.

Una vez que el programa requerido para resolver un problema ha sido preparado, puede usarse una y otra vez para la solución de problemas similares.

El objeto de este folleto es enseñar a escribir programas en el lenguaje Leal, los cuales la máquina es capaz de procesar.

2) Componentes de la CID 201-A

a) *Memoria Interna*

Es en la que se almacena la totalidad de la información recibida por la máquina o sea, los datos, las instrucciones a realizar y los resultados intermedios calculados.

La memoria de la Computadora está dividida en diferentes compartimentos llamados localizaciones. Cada localización puede contener una palabra, es decir, un dato o una instrucción, los cuales se representan según un código por un número binario. Cada localización tiene capacidad para un número binario de doce bits. Cada localización de la memoria tiene una dirección que es un número único entre 0 y 4095, que la identifica en la memoria. La CID 201-A tiene pues 4096 localizaciones decimales.

b) *Unidad de Control*

En esta parte de la Computadora es donde se controla la secuencia del programa y las operaciones aritméticas que deben realizarse.

c) *Unidad Aritmética*

Es donde se efectúan las operaciones aritméticas, bajo la dirección de la unidad de control. Está constituida fundamentalmente por el acumulador y el enlace.

d) *Memoria externa*

En la CID 201-A está compuesta por cintas magnéticas, capaces de almacenar 80 K palabras por casettes. El equipo utilizado es una grabadora portátil de audio. El acceso a los casettes no es autónomo.

e) *Unidad de entrada*

Es por donde se le introducen a la máquina los datos del programa y el programa de instrucciones. La CID 201-A admite entrada por una lectora de cinta de papel perforado de cinco canales, por una grabadora de cinta magnética. por el teclado decimal, por el teclado del teletipo y por la consola de gobierno.

f) *Unidad de salida*

Es la que se encarga de trasmitir al exterior el resultado del problema o datos parciales que después serán utilizados como entrada. La CID 201-A lo hace por el telex con el uso opcional de la perforadora de cinta de papel, por la pizarra luminosa y mediante la grabadora.

3) **Lenguaje y Compilador**

Las instrucciones de la Computadora CID 201-A son exclusivamente numéricas, o sea que la máquina sólo trabaja e interpreta números binarios. Aunque el programar en lenguaje de máquina, o sea codificando las instrucciones, es conveniente desde el punto de vista de la eficiencia en el uso de la Computadora, es muy difícil para el programador eventual su estudio y correcta y cómoda utilización. Por ello se crean los llamados lenguajes simbólicos que permiten al programador escribir sus trabajos de una manera similar a su forma de expresión acostumbrada. Además, el uso de lenguajes simbólicos, reduce considerablemente el número de instrucciones y disminuye la cantidad de errores.

Son lenguajes simbólicos el Autocode, el Fortran, el Cobol, el Algol, el Leal, etc.

Cada lenguaje tiene su vocabulario, su sintaxis y su repertorio de instrucciones permisibles, las cuales el programador debe dominar perfectamente.

Debe quedar aclarado que la máquina sólo "entiende" números binarios, por lo que hace falta un programa que nos traduzca del lenguaje simbólico al lenguaje de máquina. Esto es realizado por el programa compilador o simplemente compilador.

Los esquemas de programación, que traducen de un lenguaje a otro, suponen tres clases de programas distintos que no se deben confundir. En primer lugar el programa en lenguaje simbólico, que describe el trabajo en el lenguaje que conviene al programador conocido como programa fuente. Después el programa en lenguaje de máquina que se

obtiene a partir del programa fuente y que es llamado programa objeto. Por último el programa compilador, que opera sobre el programa fuente y da como resultado el programa objeto. Para el programa compilador, el programa fuente es simplemente, información de entrada y de acuerdo a este elabora y calcula el programa objeto.

El compilador se prepara de una vez para siempre y debe introducirse en la Computadora antes de comenzar la lectura de cualquier programa simbólico. El proceso de ejecución de cualquier trabajo particular se realiza en dos etapas: primero se traduce o compila el programa en lenguaje simbólico a su equivalente en lenguaje de máquina y después se ejecuta el programa en lenguaje de máquina. Son dos etapas separadas. La compilación se ha de realizar totalmente, antes de iniciar cualquier trabajo real.

En este curso estudiaremos el lenguaje Leal, que nos permitirá programar fácilmente para la CID 201-A.

II.—DESCRIPCION DEL LEAL

1) Representación de los números en la Computadora

Hay dos formas de representar los números en la Computadora: enteros y punto flotante. Cada número (o variable) ocupa dos localizaciones de memoria.

a) *Enteros*:

Las variables y constantes denominadas enteros son aquellas que sólo pueden tomar valores numéricos enteros, o sea los números 0, 1, 2, 3, . . *n* ó −1, −2, −3, . . −*n*. Su representación en binario en la máquina ocupa dos localizaciones (24 bits), por lo tanto estarán comprendidos entre $-8'388{,}608_{10}$ y $8'388{,}607_{10}$. Cualquier número en cuya operación la máquina se exceda de estos valores, nos indicará capacidad excedida señalándose error.

b) *Punto Flotante.*

Las variables y constantes denominadas en punto flotante son aquellas que siendo enteros o fraccionarios se representan como una fracción multiplicada por una potencia de dos. La máquina almacena la fracción o mantisa en base dos y el exponente de dos, o sea:

		La máquina almacena			
		Mantisa		*Exponente*	
Número	*Se expresa*	Signo	Número	Signo	Número
52	0.8125×2^6	0	.110100	0	00110
−1.5	-0.375×2^2	1	.01100	0	00010
20	0.625×2^5	0	.10100	0	00101
0.375	0.75×2^{-1}	0	.11000	1	00001

El 0 identifica al signo más y el 1 al signo menos.

La máquina le da de las dos localizaciones designadas al número, una y media (18 bits) a la mantisa, y media (6 bits) al exponente. Luego los números en flotante estarán comprendidos entre el rango de -2.3×10^{10} y 1.0×10^{9} aproximadamente. Cuando se excede de este rango los cálculos no son correctos señalándose error.

La precisión de los números enteros será hasta 7 dígitos decimales y en la fracción hasta 5, si la parte entera es cero, redondeando la cifra, si el sexto dígito es mayor o igual que 5.

2) **Estilo de escritura**

Las instrucciones son escritas en columna.

Los espacios (sp) y retornos del carro (cr) pueden ser escritos en cualquier momento del programa ya que son ignorados.

El número de caracteres por renglón no tiene límite.

El programa ha de iniciarse con la palabra LEAL antecedida de line feed (lf).

En el siguiente renglón se coloca el nombre del programa, el cual tiene que aparecer y no puede ocupar más de un renglón.

A continuación se ponen las declaraciones de las variables, las que se explicarán más adelante.

Y después las instrucciones del programa en el orden de ejecución. Culminando el programa con la instrucción FIN.

Para dar una idea de como es un programa en LEAL, pondremos un ejemplo donde se halla el seno, coseno y tangente de la suma de dos ángulos:

```
LEAL                                  Encabezamiento
HALLAR SENO, COSENO Y TANGENTE        Título
ENT          Declaración de las variables enteras.
FLO ABC      Declaración de las variables flotantes.
REF (1)      Declaración de los números de referencia.
LEC A        Instrucción de lectura.
LEC B        Instrucción de lectura.
A + B = C    Suma de los dos ángulos.
C SEN = A    Seno de la suma.
PER A        Impresión del seno de la suma.
C COS = B    Coseno de la suma.
PER B        Impresión del coseno de la suma.
```

B ? SSC (1)	Saltar a (1) si B = 0.
A/B = A	Tangente de la suma tan $X = \frac{\text{Sen } X}{\text{Cos } X}$
PER A	Impresión de la tangente de la suma.
(1) FIN	Fin del programa.

3) **Operandos**

Un operando es lo mismo una variable que una constante, pudiendo ser una cantidad en forma entera o en forma flotante. Luego los cuatro tipos de operandos son:

a) Variables en punto flotante.

b) Constantes en punto flotante.

c) Variables enteras.

d) Constantes enteras.

Variables

Cualquier letra del alfabeto (excepto la $\tilde{n}$) puede ser usada para representar una variable. El programador puede decidir cual variable representará un entero y cual una cantidad en punto flotante. Una vez hecha la selección, dichas variables permanecerán con esa condición durante todo el transcurso del programa, cualquier cambio será detectado como error por el compilador. Sólo pueden usarse letras mayúsculas con o sin subíndice.

Constantes

Las constantes necesarias en el cálculo deben aparecer en el programa en forma decimal y serán tratadas como enteros o flotantes de acuerdo al tipo de variable a que estén asociadas.

Si una constante entera está asociada a una variable flotante y dicha constante inicia un renglón, ésta ha de llevar punto y el primer cero después del punto.

EJEMPLOS:

A + 4 = B Si A y B son flotantes, 4 será tratado como flotante.
I + 4 = J Si I y J son enteros, 4 será tratado como entero.
4.0 + A = B Si A y B flotantes, se pone al iniciar el renglón 4.0
4 + I = J
A + 1.56 = B.

Constantes del tipo 1.25×10^{-2} no son permitidas en el cuerpo del programa, deben escribirse en la forma 0.0125.

4) **Convenio adoptado en el texto**

Para reducir la cantidad de explicación necesaria, se adoptará la siguiente convención.

A, B, C, D...	representan variables en punto flotante.
I, J, K, L...	representan variables enteras.
m, *n*	representan constantes enteras positivas.
p, *q*, *r*	representan constantes enteras, positivas o negativas.
x, *y*, *z*	representan constantes en punto flotante, positivas o negativas.

5) **Subíndices o sufijos**

Los sufijos nos permiten designar con una misma letra, a un grupo de variables que en nuestro programa tienen cierta similitud. Además sin su uso tendríamos el número de variables a usar en un programa reducido al número de letras permisibles del alfabeto o sea a 26.

Las letras o variables con sufijos serán tratadas por el compilador como variables diferentes, asignándole distintas localizaciones a cada una de ellas.

Cualquier número entero positivo o cero, así como una variable entera que no tome valores negativos puede usarse como sufijo de cualquier tipo de variable.

Los sufijos irán encerrados entre paréntesis, nunca sin estos.
La siguiente tabla muestra las formas permitidas:

Variables en Punto Flotante	*Variables Enteras*	*Tipos de Sufijos*
A ó A (0)	K ó K (0)	Simple
A (n)	K (n)	Simple
A (I)	K (I)	Simple
A (I (n))	K (I (n))	Complejo.

No se permiten operaciones en los sufijos.

Los subíndices o sufijos nos brindan la facilidad de establecer un programa para efectuar cálculos básicos, efectuando después los mismos cómputos, simplemente, cambiando el valor de los subíndices.

EJEMPLO: Supongamos que necesitamos calcular la suma de los cuadrados de 20 números, desde X(1) hasta X(20) que se encuentran almacenados en la computadora. Podemos entonces establecer una proposición aritmética con relación al elemento X(I) sujetándonos después a las normas para que I tome todos los valores desde 1 hasta 20.

Este ejemplo lo veremos resuelto más adelante cuando se vean las instrucciones de ciclo.

6) **Paréntesis**

Los paréntesis sólo son permitidos para encerrar sufijos y números de referencia, no estando permitido su uso para señalar prioridad en las operaciones. Otro uso de los paréntesis, que se verá más adelante, es para observaciones y chequeo.

(A + B) / C No es permitido.

7) **Números de referencia**

Teniendo en cuenta que la Computadora efectúa los pasos de un cálculo en la secuencia en que aparecen las instrucciones y por esta razón no es necesario numerar todas las instrucciones de un programa, sin embargo hay ocasiones en que es conveniente que la Computadora rompa la secuencia normal y salte a otra parte del programa. Es posible indicarle a la máquina cual instrucción del programa debe obedecer primero y cuando debe detenerse, esto, así como otras facilidades pueden lograrse con el uso de los números de referencia.

Al escribir el programa los números de referencia aparecerán al comenzar el renglón, a la izquierda y entre paréntesis.

Los números de referencia deben ser siempre constantes enteras y positivas.

No es permitido utilizar el mismo número de referencia a la izquierda de más de una instrucción del programa.

Posteriormente se verán otros usos de los números de referencia.

EJEMPLO:

(1) A + B = C.

8) **Declaraciones**

Para permitir al compilador LEAL la transcripción correcta de un programa, es necesario que reciba previamente las letras que se van a utilizar en éste ya sea como variables enteras o como variables en punto flotante, así como los números de referencia que se van a emplear. El compilador reserva dos localizaciones por cada variable y una por cada número de referencia.

Se declara la variable con el mayor subíndice conque aparece en el programa, quedando así declaradas automáticamente todas las anteriores. Ejemplo, si se declara A (5), se han declarado en total 6 variables, que son: A (0), A (1), A (2), A (3), A (4) y A (5).

De forma similar sólo se declara el mayor número de referencia que aparece en el programa.

Ninguna variable puede aparecer dos veces en la declaración.

Las declaraciones son:

ENT Declaración de variables enteras.
FLO Declaración de variables flotantes.
REF Declaración de los números de referencia.

Se colocan al inicio del programa, inmediatamente después del título y en el orden anterior.

ENT

La instrucción ENT especifica las letras que serán utilizadas como variables enteras. Las letras pueden declararse en cualquier orden y no van separadas por comas; pueden o no llevar espacios entre ellas, pues estos siempre son ignorados por el compilador.

Esta instrucción no puede omitirse en el programa, aunque no aparezcan variables enteras en el programa en ese caso se pone ENT y a continuación (cr–lf).

FLO

La instrucción FLO especifica las letras que serán utilizadas como variables en punto flotante.

Para ella se cumple lo mismo que para la instrucción ENT.

Una variable no puede ser declarada como entera y como flotante en un mismo programa. Una variable declarada en ENT (respectivamente FLO), será considerada como entera (respectivamente flotante) en todo el programa, o sea, la declaración no puede variarse en el transcurso del programa.

REF

La instrucción REF especifica los números de referencia que aparecen en el programa. Tras esta palabra se coloca el mayor número de referencia que aparece, entre paréntesis.

Ejemplo de declaraciones:

```
LEAL
TITULO
ENT I (3) K J L (2)
FLO B (2) C
REF (3)
```

Están declaradas como variables enteras: I(0), I(1), I(2), I(3), K(0), J(0), L(0), L(1) y L(2).

Están declaradas como variables flotantes: B(0), B(1), B(2) y C(0).
Están declarados como números de referencia: (1), (2) y (3).

No permitido	*Permitido*
ENT I(n)	ENT I(5)
FLO A(m)	FLO A(18)
REF (p)	REF (2)

Facilidad Común:

La facilidad común consiste en identificar una variable con sufijo mayor que el declarado con una variable del mismo tipo declarada a continuación de esta. Veamos esto con un ejemplo.

ENT I(3) J(2) K.

Tendremos la siguiente identificación:

I (4) = J (0)
I (5) = J (1)
I (6) = J (2)
I (7) = J (3) = K (0).

Es decir, que si en el programa aparece I(4), la máquina toma a J(0), etc. Si en este ejemplo aparece I(n) con *n* mayor que 7 se detecta error. También es permitido: L = — 3 y si aparece K(L), se refiere a J(0). Esto sólo se puede hacer con sufijos variables. Si en este ejemplo L es menor que —7, K(L) toma un valor arbitrario sin detectarse error. Por lo que no es recomendable el uso de variables negativas. Todo lo anterior se cumple igual si I, J y K fueran flotantes.

Si bien, el orden en las declaraciones de las variables es arbitrario, una vez establecido éste, se debe tener en cuenta para el correcto empleo de la facilidad común.

9) **Instrucciones de entrada y salida**

a) *Instrucción de entrada.*

La instrucción de entrada en el LEAL es LEC, colocada al principio del renglón y seguida de una, y sólo una, variable entera o flotante, sin, o con cualquier tipo de sufijo.

Flotantes	*Enteras*
LEC A	LEC K
LEC A (n)	LEC K (n)
LEC A (I)	LEC K (I)
LEC A (I (n))	LEC K (I (n))

La instrucción:

LEC A significa leer el valor que aparece en la cinta de datos (o el que se le da por teclado de telex, etc.) en código Murray y darle este valor a la variable A.

b) *Instrucciones de salida.*

La instrucción de salida en el LEAL es PER, colocada al principio del renglón y seguida de una, y sólo una, variable entera o flotante, sin o con cualquier tipo de sufijos.

Flotantes	*Enteras*
PER A	PER K
PER A (n)	PER K (n)
PER A (I)	PER K (I)
PER A (I (n))	PER K (I (n))

La instrucción:

PER A significa imprimir el contenido de la variable A.

Nota: La forma en que se imprime este valor será estudiada en el epígrafe 19. En los ejemplos que aparecen antes de dicho epígrafe se supondrá que los resultados se imprimen en el telex en columna, o sea, uno en cada renglón. Una vez estudiadas las formas de impresión deben añadirse a estos ejemplos las instrucciones que permitan una correcta impresión.

Opcionalmente, puede perforarse una cinta de papel.

En las instrucciones de entrada y salida:

No permitido	*Permitido*
LEC AB	LEC A
	LEC B
	PER A
PER AB	PER B

Cintas de datos.

Los programas son escritos en forma general; para resolver un problema concreto se le introducen los datos por una cinta perforada teniendo en cuenta el orden en que son leídos en el programa.

Los datos en la cinta deben ir separados por tres espacios.

Los datos se pueden dar también por el teclado del teletipo.

10) **Instrucciones de Alto y Fin**

ALT Es la instrucción de alto en el LEAL, hace que la computadora pare la ejecución del programa objeto.

En un mismo programa pueden aparecer varias instrucciones ALT.

FIN Es la instrucción que detiene el compilador durante la traducción del programa fuente y hace que la computadora pare durante la ejecución del programa objeto. Antes de FIN no hay que poner ALT, ya que en la ejecución del programa objeto FIN implica ALT.

En un programa sólo podrá aparecer una instrucción FIN.

11) Operadores y operaciones aritméticas

Operadores

Los operadores son distintos símbolos que colocados entre los operandos indican operaciones aritméticas y se representan:

+	Suma
−	Resta
:	Multiplicación
/	División.

Los operadores +, −, : pueden ser usados tanto para operaciones con enteros como con flotantes.

El operador / sólo puede utilizarse para operaciones en punto flotante.

Operaciones Aritméticas

	Flotantes	*Enteros*
a) Fijar el valor	A = C	I = K
b) Suma	A + B = C	I + J = K
c) Resta	A − B = C	I − J = K
d) Multiplicación	A : B = C	I : J = K
e) División	A / B = C	

a) Estas instrucciones significan evaluar la expresión de la izquierda y fijar ese valor en la variable que está a la derecha.

b) A y B pueden sustituirse por constantes, por C o ser idénticas.

EJEMPLO: A : 2 = A
A + A = A

Son dos formas de duplicar el contenido de A.
A + 1 = A

Incrementa en uno el contenido de A.

Lo mismo ocurre con I, J y K.

c) El signo menos (−) se puede utilizar al comenzar el renglón.

EJEMPLO: − A = A

Cambia el signo al contenido de A.

Lo mismo ocurre con I y J.

d) Las operaciones aritméticas sólo podrán realizarse entre variables del mismo tipo, o sea, flotante con flotante depositada en flotante o entera con entera depositada en entera.

e) La división en enteros no está permitida, solamente en flotante, siendo necesario para dividir cantidades enteras, expresarlas en forma flotante, como se aprenderá más adelante.

Como datos técnicos que el programador debe conocer para la optimización de su programa tenemos:

a) Las operaciones en enteros son más rápidas que las operaciones en punto flotante.

b) La suma y la resta son más rápidas que la multiplicación y la división.

En el lenguaje Leal no hay límite en el número de operaciones a realizar en el mismo renglón.

Siempre se realiza la operación aritmética que indica el operador, entre la expresión aritmética que está a la izquierda y la variable o constante que esté a la derecha, o sea el operador que está a la izquierda prevalece siempre sobre los otros, cualesquiera que éstos sean. La máquina opera, pues de izquierda a derecha, comenzando por el inicio del renglón en el orden en que están colocadas las operaciones y fijando el valor obtenido en la variable que tras el signo igual concluye esta instrucción.

EJEMPLO:

A+B−C: D−E / F = G donde A, B, C, D, E, F y G son flotantes.

Significa:

$$G = \frac{(A + B - C)\, D - E}{F}$$

Por tanto pueden realizarse varias operaciones en una sola instrucción, aunque debido al orden en que se realizan las operaciones y a la imposibilidad de utilizar los paréntesis para señalar preferencias en las operaciones, ciertos cálculos tenemos que realizarlos en varias instrucciones.

EJEMPLO: Calcular el producto escalar de los vectores A y B.

$A = (A_1\ A_2\ A_3)$ $A_1\ A_2\ A_3\ B_1\ B_2\ B_3$ flotantes

$B = (B_1\ B_2\ B_3)$

Sabiendo que $A.B = A_1 B_1 + A_2 B_2 + A_3 B_3$.

La programación de este cálculo será:

A_1: $B_1 = A_1$
A_2: $B_2 = A_2$
A_3: $B_3 + A_1 + A_2 = A_1$
PER A_1.

Si lo programamos como sigue:
A_1: $B_1 + A_2$: $B_2 + A_3$: $B_3 = A_1$.

La máquina lo que nos haría es:

$((A_1 B_1 + A_2) B_2 + A_3) B_3 = A_1$ ó sea
$A_1 B_1 B_2 B_3 + A_2 B_2 B_3 + A_3 B_3 = A_1$ que no es lo que deseamos.

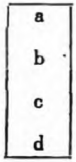
Luego la posibilidad que da el Leal de poner varias instrucciones en una debe ser aprovechada al máximo por el programador, pero siempre teniendo en cuenta su correcto uso.

EJEMPLO 1 Operaciones Aritméticas.

Hacer un programa en Leal para hallar el volumen de la figura.

VER FIGURA No. 1

```
LEAL
HALLAR EL VOLUMEN
ENT
FLO A B C D
REF (1)
LEC A
LEC B
LEC C
LEC D
B / 2 + C : A : D = D
PER D
FIN
```



Cinta de datos

NOTA: En este programa no se utilizan números de referencia, pero siempre es necesario declarar al menos uno.

12) **Funciones**

Las instrucciones de funciones son de la forma:

F FUNCION = A

Que significa evaluar la expresión de la izquierda y luego fijar la variable de la derecha con ese valor.

El signo menos (−) puede ir antes de la expresión, o sea:
−F FUNCION = A

Las funciones con sus respectivos símbolos son:

SEN	Seno
COS	Coseno
ACT	Arcotangente
LOG	Logaritmo en base 10
LNE	Logaritmo en base e
EXP	Exponencial
RAC	Raíz cuadrada
PEN	Parte entera
FRA	Parte fraccionaria
NOR	Normalizar, o sea llevar un número entero a flotante
EAB	Módulo de una variable entera
FAB	Módulo de una variable flotante.

EJEMPLOS:

F SEN = B (F y B flotantes).
F COS = B (F y B flotantes)
F ACT = B (F y B flotantes)
F LOG = B (F y B flotantes)
F LNE = B (F y B flotantes)
F EXP = B (F y B flotantes)
F RAC = B (F y B flotantes)
F PEN = J (F flotante y J entera)
F FRA = B (F y B flotantes)
J NOR = B (J entera y B flotante)
J EAB = I (J e I enteros)
F FAB = B (F y B flotantes)

Aclaraciones:

JNOR = B Significa tomar la variable entera J, convertir el contenido a flotante y colocar en B ese resultado. B tiene que ser declarada obligatoriamente como flotante.

F PEN = J Significa hallar la parte entera a la variable flotante F y colocar en J ese resultado. J tiene que ser declarada obligatoriamente como entera.

B FRA = A Significa hallar la parte fraccionaria a la variable flotante B y colocar ese resultado en la variable flotante A.

La máquina toma la parte entera de un número como el *menor* número entero más próximo a él y la parte fraccionaria es el complemento con respecto al número de la parte entera, es decir:

PARTE ENTERA DE A + PARTE FRACCIONARIA DE A = A.

EJEMPLO:

Número	*Parte entera*	*Parte Fraccionaria*
2.1	2	0.1
3	3	0
− 3.4	− 4	0.6
− 0.1	− 1	0.9

En el lenguaje Leal las funciones que se van a emplear en un programa no tienen que ser declaradas.

13) Expresiones aritméticas

Las expresiones aritméticas son secuencias cualesquiera de operaciones donde se utilizan operandos, operadores y funciones.

Una expresión aritmética puede ser una constante, una variable, una función evaluada en una constante o en una variable, una operación aritmética, o la combinación de cualquiera de ellas mediante un operando o mediante otra función.

EJEMPLO:

4.1 es una expresión aritmética.
A es una expresión aritmética.
I(J(3)) es una expresión aritmética.
A SEN es una expresión aritmética.
A SEN COS es una expresión aritmética.
A SEN COS RAC es una expresión aritmética.
A − B SEN − C ACT LNE: D es una expresión aritmética.

Una expresión aritmética completa, va seguida de un signo igual (=) y de una variable en la que se fija el valor de esta, o de un signo de interrogación para señalar un salto como veremos más adelante.

EJEMPLO 2 Funciones

Hacer un programa en Leal para evaluar la función $f(x)$ en un X $X \in (1, 1.5)$

$$f(X) = \frac{\text{Exp}(X+\tan X) + \sqrt{\ln X}}{2 + \text{Arctg. } X^2}$$

```
LEAL
EVALUAR F (X)
ENT
FLO  X Y Z
REF (1)
LEC X
X: XACT + 2 = Z
XCOS = Y
XSEN / Y + X EXP = Y
XLNERAC + Y / Z = Y
PERY
FIN
```



Cinta de datos

EJEMPLO 3

Hallar el área de la figura conociendo la hipotenusa a y el ángulo α

VER FIGURA No. 2

```
LEAL                    a ~ A
HALLAR EL AREA          α ~ B
ENT
FLO  A B H
REF (1)
LEC  A
LEC  B
BSEN: A = H
BCOS: A = B
H: B/2 = B
H: H: 3.1416 / 8 + B = B
PER B
FIN
```

14) **Instrucciones de salto**

a) *Salto incondicional*

Viene dado por la instrucción.

VEA (n) y significa romper la secuencia normal del programa y obedecer la instrucción cuyo número de referencia es (n) y continuar en secuencia desde allí.

b) *Salto condicional*

Viene dado por la forma

E.A. ? CONDICION DE SALTO (n)
Donde E.A. es una expresión aritmética cualquiera.

Estas instrucciones rompen la secuencia normal del programa si se cumple la condición de salto, saltando a la instrucción precedida del número de referencia (n) y continuando en secuencia desde allí.

Condiciones de Saltos

SSP	Salte si es positivo	(si > 0)
SSN	Salte si es negativo	(si < 0)
SSC	Salte si es Cero	(si $= 0$)
SNP	Salte si no es positivo	(si $\leqslant 0$)
SNN	Salte si no es negativo	(si $\geqslant 0$)
SNC	Salte si no es Cero	(si $\neq 0$)

EJEMPLOS:

A? SSP (3)	Si A es positivo salta a (3), sino continúa la secuencia normal.
A SEN EXP – B? SSN (4)	Si la expresión aritmética es negativa salta a (4), sino, continúa la secuencia normal.
BEXP – C/D? SSC (1)	Si la expresión aritmética es cero salta a (1), sino continúa la secuencia normal.
A: D/C – A? SNP (2)	Si la expresión aritmética no es positiva (o sea es $\leqslant 0$) salta a (2,) sino, continúa la secuencia normal.
INOR + A? SNN (8)	Si la expresión aritmética no es negativa (o sea es $\geqslant 0$) salta a (8), sino, continúa la secuencia normal.
APEN – K? SNC (9)	Si la expresión aritmética no es cero (o sea es >0 ó <0) salta a (9) sino, continúa la secuencia normal.

EJEMPLO 4

Hacer el programa para hallar la solución de una ecuación de segundo grado.

$a\,x^2 + b\,x + c = 0$ si $X = \dfrac{-b \pm \sqrt{b^2 - 4\,ac}}{2a}$

$a \sim A$
$b \sim B$
$c \sim C$

LEAL
SOL. DE UNA ECUACION DE SEGUNDO GRADO

```
ENT
FLO A B C D
REF (2)
LEC A
LEC B
LEC C
B: B = D
−4.0: A: C + D = C
C ? SSN (1)
C ?.SSC (2)
C (o) RAC = C
− B − C: 0.5/A = D
PER D
(2) − B + C: 0.5/A = B
PER B
(1) FIN
```

15) **Instrucciones de Ciclo**

La instrucción CIC (ciclo) permite la ejecución repetida de una serie de instrucciones. A esta repetición se le llama lazo.

Para cada instrucción CIC debe haber una y sólo una instrucción REP (repita).

Antes de comenzar una instrucción cíclica, deberá habérsele asignado a la variable de control o de conteo un valor por medio de una expresión aritmética cualquiera. Dicha variable tomará diferentes valores dentro del lazo.

Es necesario asignar números de referencia a las instrucciones CIC y REP, pero su posición difiere en ambos. En la instrucción CIC va antes de ella y en la instrucción REP después de la misma.

Forma general

(Expresión Aritmética) = I	Asignación de valor a la variable de conteo.
(n) CIC	Comienzo del ciclo
.	
.	
.	
.	
.	**Lazo**
.	
.	
.	
.	
REP (n) I	Instrucción de repetir, contiene siempre a la variable de conteo y otras variables o constantes como veremos a continuación.

En Leal hay tres tipos de ciclos, cuya caracterización viene dada por la instrucción REP, o mejor dicho, por la colocación de las variables o constantes que se ponen tras el número de referencia que sigue a la palabra REP.

Todas las variables pueden utilizar cualquier tipo de subíndice permitido.

Se puede poner un ciclo dentro de otro todas las veces que se necesite, teniendo siempre el cuidado de poner los números de referencia de cada uno correctamente y cerrarlos adecuadamente.

También el número de ciclos dentro de un programa es ilimitado. sean o no independientes.

Tipos de Ciclo

1) 1er. tipo o ciclo lista

```
(n)  CIC
      .
      .
      .
      .
      .
      .
      .
      .
REP (n) I'J'K'L
```

Este tipo de ciclo consiste en que el lazo se efectúa tantas veces como variables hayan en total (en este caso 4). La variable de conteo I entra al ciclo efectuando el primer lazo con el valor que se le haya asignado anteriormente y luego toma seguidamente los valores de las variables restantes.

En este caso se efectuará el ciclo de la siguiente forma:

1er. lazo: I será igual a I_0 (valor asignado anteriormente)
2do. lazo: I será igual a J
3er. lazo: I será igual a K
4to. lazo: I será igual a L

Una vez repetido el ciclo, tantas veces como variables aparezcan en el REP, de la forma especificada, el programa seguirá la secuencia normal.

La variable de conteo puede ser entera o flotante. En el primer caso, las restantes puden ser constantes enteras positivas o variables enteras; sin embargo si la variable de conteo es flotante, el resto pueden ser constantes positivas o variables flotantes. Si hay que hacer el ciclo con

una constante negativa, hay que fijar previamente su valor en una variable, y hacer el ciclo con la susodicha variable. Pueden ser tantas variables como sean necesarias.

Este tipo de ciclo se caracteriza porque todas las variables o constantes van separadas por medio de un apóstrofo.

EJEMPLO 5

Evaluar un polinomio por el método de Horner para los siguientes valores.

$$X = -8.5, 2, 3.87$$

$$P(X) = A_0 X^n + A_1 X^{n-1} + \ldots + A_{n-1} X + A_n$$
$$= ((\ldots(((A_0 X + A_1)X + A_2)X + A_3)X + \ldots + A_{n-1})X + A_n$$

Datos: N, A., A_1 A_n

```
LEAL
EVALUACION DE UN POLINOMIO EN VALORES ARBITRARIOS
ENT NI
FLO A (100) D X
REF (4)
LEC N
0 = I
(1) LEC A (I)
I + 1 = I
I - N - 1 ? SNC (1)
- 8.5 = X
(2) CIC
0.0 = D
0 = I
(4) D + A (I) = D
N - I? SSC (3)
D: X = D
I + 1 = I
VEA (4)
(3) PER D
REP (2) X'2'3.87
FIN
2 -) 2° tipo de Ciclo
(n) CIC
        .
        .
        .
        .
        .
        .
        .
        .
REP (n) I, J, K
```

Este tipo de ciclo consiste en que el lazo se efectúa primero con el valor que previamente se le ha asignado a la variable de conteo I, luego con este valor incrementado en J, luego con el anterior incrementado en J de nuevo, repitiéndose hasta que alcanza el valor K, si no lo alcanza exactamente, la última vez lo hace con un valor menor que K, es decir, el lazo nunca se hace con un valor mayor que K.

Luego:

1er. lazo I será igual a I_0 (valor asignado anteriormente)
2do. lazo I será igual a $I_0 + J$
3ro. lazo I será igual a $I_0 + 2J$

.
.
.

N° lazo I será igual a $I_0 + (N-1)J$
siendo $I = I_0 + (N-1)\ J \leqslant K$, luego se hace N veces, donde

$$N \leqslant \frac{K - I_0}{J} + 1$$

Si el valor inicial I es mayor que K, no se repite, efectándose una sola vez con este valor.

La variable de conteo I puede ser entera o flotante. En el primer caso, J y K son enteros, o pueden ser sustituídos por constantes enteras positivas; sin embargo si la variable de conteo I es flotante, J y K son flotantes, o pueden ser sustituídas por constantes positivas.

En este tipo de ciclo intervienen sólo tres variables o constantes (nótese que la primera es necesariamente una variable que es la de conteo).

Las variables o constantes están separadas por comas.

EJEMPLO 6

Hacer el programa del cálculo de la media y la desviación típica sabiendo que

$$\bar{X} = \frac{\sum_{i=1}^{N} x_i}{N} \qquad S = \sqrt{\frac{\sum_{i=1}^{n} (x_i - \bar{x})^2}{N}}$$

```
LEAL
CALCULO DE LA MEDIA Y LA DESVIACION TIPICA
ENT     IN
FLO     X(400)   ABY
REF   (2)
LEC   N
1 = I
0.0 = X
(1)  CIC
LEC X (I)
X (I) + X = X
```

```
REP (1) I, 1, N
N NOR = A
X/A = X
PER X
1 = I
0.0 = Y
(2) CIC
X (I) − X = B
B: B + Y = Y
REP (2) I, 1, N
Y/A RAC = Y
PER Y
FIN
```

EJEMPLO 7

Ejemplo planteado en el epígrafe 5.

Hacer el programa de la suma de los cuadrados de 20 números, desde X (1) hasta X (20) que se encuentran almacenados en la computadora.

```
LEAL
SUMA DE CUADRADOS DE 20 NUMEROS
ENT I
FLO X (20)
REF (1)
0.0 = X
I = I
(1) CIC
X (I): X(I) + X = X
REP (1) I, 1, 20
PER X
FIN
```

No hay que leer los números, pues se encuentran en la máquina.

NOTA: El proceso de lectura empleado en el ejemplo 5, o sea, por saltos puede programarse mejor con el segundo tipo de ciclo, quedaría.

```
0 = I
(1) CIC
LEC A (I)
REP (1) I, 1, N
3 =) 3er. tipo de ciclo
(n) CIC
      .
      .
      .
      .
      .
      .
      .
REP (n) I, J' K
```

Este tipo de ciclo consiste en que el lazo se efectúa primero con el valor que previamente se le ha asignado a la variable de conteo I, luego con este valor incrementado en J, luego con el anterior incrementado en J de nuevo, repitiéndose hasta hacerse el ciclo K veces.

1er. lazo I será igual a I. (valor asignado anteriormente)
2do. lazo I será igual a I. + J
3ro. lazo I será igual a I. + 2 J

K lazo I será igual a I. + (K − 1) J

Si K = 1 no se repite, efectuándose una sola vez con el valor que trae la I.

La variable de conteo I, puede ser entera o flotante. En el primer caso la J será entera o constante entera positiva; en el segundo caso J será flotante o constante positiva. K siempre será una variable entera o una constante entera positiva.

Al igual que en los casos anteriores, si J debe ser una constante negativa, hay que fijar previamente su valor en una variable y hacer el ciclo con esta variable.

En este tipo de ciclo intervienen sólo tres variables o constantes (nótese que la primera es necesariamente una variable, que es la de conteo).

La variable de conteo se separa de la variable o constante de incremento por una coma y esta de la variable o constante que señala las veces a ejecutarse el lazo por un apóstrofo.

EJEMPLO 8 Ciclos

Hacer el programa para hallar la transpuesta de una matriz A*mn* leída por filas

$$A = \begin{bmatrix} a1 & a2 & \dots & an \\ an+1 & an+2 & & a2n \\ \vdots & & & \\ a(m-1)n+1 & \dots & & amn \end{bmatrix}$$

$$\text{y } A^t = \begin{bmatrix} a1 & an+1 & \dots & a(m-1)n+1 \\ a2 & an+2 & \dots & a(m-1)n+2 \\ \vdots & \vdots & & \vdots \\ an & a2n & & amn \end{bmatrix}$$

```
LEAL
HALLAR LA TRANSPUESTA DE UNA MATRIZ
ENT  I J M N P
FLO  A (500)
REF (3)
LEC M
LEC N
M: N = P
1 = I
(1) CIC
LEC A(I)
REP (1) I, 1'P *
1 = J
(2) CIC
0 = I
(3) CIC
I + J = P
PER A (P)
REP (3) I, N' M
REP (2) J, 1' N * *
FIN
```

Nota: * Sería equivalente poner:

```
REP (1) I, 1, P
```

* * sería equivalente poner:

```
REP (2) J, 1, N
```

EJEMPLO 9

Imprimir $V = 1.8L \tanh \frac{6.3}{L} D$ para

L = 270, 275, 280, 285, 290
D = 30, 31, 32, 33, 34

Sabiendo $\tanh X = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ Tanh (tangente hiperbólica).

```
LEAL
HALLAR V
ENT
FLO  D L K F V
REF (1)
30.0 = D
270.0 = L
(1) CIC
6.3: D/L = K
- K EXP = F
K EXP = K
K + F = V
```

K − F/V: 1.8: L = V
PER V
D + 1 = D
REP (1) L, 5, 290
FIN

16) **Subrutinas**

En muchos programas es necesario realizar un grupo de instrucciones repetidas veces, las cuales constituyen una operación independiente dentro del programa principal, para esto se usan las subrutinas; las cuales no son más que programas anexos al programa principal. Por ejemplo, si en un programa determinado, hace falta resolver una ecuación de segundo grado varias veces, sería muy engorroso intercalar cada vez el programa relativo a dicha operación, siendo posible mediante el uso de las subrutinas ir, cuando se necesite, al programa de resolución de dicha ecuación, planteado una sola vez, con distintos parámetros de entrada y salida.

Es posible colocar una subrutina dentro de otra.

Las subrutinas se colocan generalmente al final del programa.

Se pueden utilizar las subrutinas un número ilimitado de veces.

En Leal las subrutinas tienen asociadas tres instrucciones, estas son VAS, CRU y FRU.

1. Una subrutina puede usarse (o llamarse) escribiendo la instrucción VAS (n) y a continuación las variables o constantes de entrada y las variables de salida, todas van separadas una de otra por una coma, sin especificarse cuales son de entrada y cuales de salida, pero sí colocando primero las de entrada y luego las de salida.

Las variables o constantes de entrada son aquellas que entregarán su contenido a otras variables en el inicio de la subrutina propiamente dicha. Estas últimas deben ser del mismo tipo (enteras o flotantes) del que son las variables o constantes de entrada, aunque pueden haber combinaciones entre flotante y enteros como variables de entrada, coincidiendo en la misma forma con las que recibirán los valores en el inicio de la subrutina.

Las variables de salida son aquellas que recibirán su contenido de las variables que contienen los resultados deseados de la subrutina utilizada. Guardan las mismas restricciones que las variables de entrada.

La instrucción VAS (n), no forma parte de la subrutina sino que es utilizada dentro del programa principal u otra subrutina, para requerir su utilización.

EJEMPLO:

VAS (n) A, N, 7.1, D (0), D (1), D(2).

Nótese que si se han de usar D (0), D (1), ... hasta D (n) es necesario colocarlas todas; no basta poner D(n).

En el Vas (n) las constantes que se pongan tienen, si son flotantes, que llevar el punto y por lo menos la primera cifra decimal.

Deben aparecer tantas instrucciones VAS (n) como veces se quiera realizar el proceso que encierra dicha subrutina, variando las variables o constantes de entrada y las variables de salida en cada caso.

2. Para identificar el comienzo de una subrutina utilizamos la instrucción:

(n) CRU y a continuación las variables receptoras.

Las variables receptoras son aquellas que recibirán su contenido de las variables o constantes de entrada por orden de coincidencia con las referidas por la instrucción VAS (n).

Estas variables receptoras tienen que estar incluidas en la declaración inicial del programa coincidiendo con el tipo (entera o flotante) de las variables o constantes de entrada.

Las variables receptoras no van separadas por comas.

EJEMPLO:

(n) CRU F(0) J F(1).

Nótese que al igual que en VAS (n) hay que poner todas las variables, pero las comas no se ponen.

Cada subrutina tiene asociada una y sólo una instrucción (n) CRU.

3. Para indicar el final de una subrutina, utilizamos la instrucción

FRU (n) y a continuación las variables emisoras.

Las variables emisoras son aquellas que emitirán su contenido a las variables de salida por orden de coincidencia con las referidas por la instrucción VAS (n). Cumplen las mismas indicaciones señaladas para las variables receptoras.

EJEMPLO:

FRU (n) H(6) H(9) 0

Nótese que al igual que en VAS (n) y (n) CRU hay que colocarlas todas, en este caso tampoco llevan comas.

Tras la instrucción FRU (n), la máquina regresa a la instrucción VAS (n) que causó la entrada a la subrutina para seguir de aquí la secuencia del programa principal.

Cada subrutina tiene asociada una, y sólo una, instrucción FRU (n).

Tanto en VAS (n), como en (n) CRU y FRU (n) las variables pueden llevar cualquier tipo de subíndice permitido.

La suma del número de variables receptoras y del número de variables emisoras ha de ser igual al número de variables o constantes de entrada o salida declaradas en el VAS (n).

Cada subrutina tiene, pues, una instrucción (n) CRU y una instrucción FRU (n) con el mismo número de referencia (n) y variables receptoras y emisoras fijas, respectivamente.

En el programa principal pueden aparecer varias instrucciones VAS (n) con el mismo número de referencia (n) y con variables de entrada y de salida que varían en cada caso.

Tanto en el CRU como en el FRU no pueden aparecer constantes, solamente variables.

Un sólo VAS (n) iría una sola vez a la subrutina. En este caso no es recomendable el uso de las subrutinas.

EJEMPLO:

```
LEAL
TITULO
ENT N J
FLO A D (2) E (7) F (1) H (10) 0
REF (4)
  .
  .
  .
  .
VAS (4) A, N, 7.1, D (0), D (1), D (2)
PER D (0)
PER D (1)
PER D (2)
  .
  .
  .
  .
  .
  .
  .
  .
ALT
(4) CRU F (0) J F (1)
  .
  .
  .
FRU (4) H (6) H (9) 0
FIN
```

Programa Principal

Subrutina

Al llegar la máquina a la instrucción VAS (4), va a la subrutina (4) y pone A en F (0), N en J y 7.1 en F (1) (nótese que A y F (0) son flotantes, N y J son enteros y 7.1 y F (1) flotantes).

Ejecuta las instrucciones señaladas en la subrutina con estos valores. Y al llegar al FRU (4), regresa al programa principal poniendo H (6) en D(0), H(9) en D (1) y 0 en D (2). Continuando la ejecución del programa principal.

También se pueden declarar las variables en (n) CRU y FRU (n) como Y (I, m), donde I es una variable entera y m una constante, teniendo que ser $I + 1 \leqslant m$ cada vez que vaya a la subrutina, si ocurre lo contrario será error; en lugar de la I, también podría ir una constante, ejemplo A(n, m) donde $n + 1 \leqslant m$.

Y (I, m) se cuenta como una sola variable a los efectos de que la suma del número de variables declaradas en (n) CRU y FRU (n) es igual al número de las declaradas en VAS (n).

Y Y(I, m) significa, que se utilizan desde Y(0) hasta Y(I), tomando o dando (según aparezca en el (n) CRU o en FRU (n)) el primer valor Y(0) a la primera variable correspondiente declaradas en el VAS (n) y las restantes de acuerdo a la secuencia de las declaraciones del programa principal.

EJEMPLO:

```
LEAL
TITULO
ENT I J ...
FLO A (10) B C Y Z X (6)
REF (1)
  .
  .
  .
  .
  .
  .
5 = I
3 = J
VAS (1) A(I), B, C
  .
  .
  .
  .
  .
  .
  .
  .
  .
```

```
ALT·
(1) CRU X (J,7)
 .
 .
 .
FRU (1) X Z
FIN
Pone A (5) en X (0)
Pone A (6) en X (1)
Pone A (7) en X (2)
Pone A (8) en X (3)
Y al salir
Pone X en B
Pone Z en C
```

Si al entrar en la subrutina J > 6 será error.

Cuando en la declaración el Index es menor que el utilizado en (n) CRU ó FRU (n), se toman las variables que le siguen a la derecha en la declaración del programa principal, hasta tomar todas las necesarias.

EJEMPLO:

```
LEAL
TITULO
ENT I...
FLO SPAB (2) CD...
REF (1)
 .
 .
 .
 .
 .
 .
VAS (1) A, B
 .
 .
 .
 .
 .
ALT
(1) CRU X
 .
 .
 .
 .
 .
 .
 .
```

Supongamos que al salir de este paso la I tiene el valor 4.

```
FRU (1) Y(I, 8)
FIN
```

Al entrar:

Pone A en X

Al salir:

```
Pone Y (0) en B (0)
Pone Y (1) en B (1)
Pone Y (2) en B (2)
Pone Y (3) en C
Pone Y (4) en D
```

Este tipo de índice se emplea generalmente en el FRU (n), cuando no se sale siempre con el mismo número de resultados.

EJEMPLO 10

Hacer el programa para hallar las combinaciones de N elementos tomados P a P.

$$C_p^n = \frac{N!}{P!(N-P)!}$$

Empleemos una subrutina para hallar los factoriales.

```
LEAL
HALLAR LAS COMBINACIONES DE N ELEMENTOS TOMADOS P A P
ENT M N O P Q T
FLO A B C
REF (2)
LEC N
LEC P
N − P = Q
VAS (1) N, N
VAS (1) P, P
VAS (1) Q, Q
NNOR = A
PNOR = B
QNOR = C
A/B/C/ = A
PER A
ALT
(1) CRU M
1 = 0
1 = T
(2) CIC
T : 0 = 0
REP (2) T, 1, M
FRU (1) 0
FIN
```

17) **Observaciones:**

En un programa en Leal se pueden intercalar observaciones que guíen al programador o al operador, estas se colocan, tras una letra 0 entre paréntesis, al inicio del renglón. Ese renglón es ignorado por el compilador.

EJEMPLO:

(0) HALLEMOS AHORA LA OTRA RAIZ DE LA ECUACION

18) **Chequeo**

En muchos programas, sobre todo si son largos, es conveniente chequear un resultado intermedio, mediante algunas instrucciones complementarias.

En el lenguaje Leal este chequeo se hace colocando (Cn) al comienzo de renglón, lo que significa que si al llegar a esta instrucción el registro de interruptores (los 12 interruptores que están en la consola) está en cero, se realizan las instrucciones que aparecen a continuación, pero si no está en cero, salta a la instrucción que tiene el número de referencia (n).

Este mecanismo permite al operador realizar o no el chequeo puesto por el programador en el programa.

EJEMPLO:

```
(C 2)  A − B − 2 = D
  .
  .                                  Instrucciones de chequeo
  .
  .
  .
  .
(2)                                  Continuación del programa.
```

EJEMPLOS GENERALES

I. Dada una lista de números reales distintos de cero (no se sabe cuantos) Hallar:

1.) La suma de los positivos y contarlos.
2.) La suma de los cuadrados de los negativos y contarlos.
3.) El total de los números.

El final de la lista se indica con un cero.

```
LEAL
HALLAR A, B, I, J, I + J
ENT I J
FLO ABX
REF (3)
0.0 = A
```

```
0.0 = B
0 = I
0 = J
(3) LEC X
X ? SSC (1)
X ? SSN (2)
X + A = A
I + 1 = I
VEA (3)
(2) X: X + B = B
J + 1 = J
VEA (3)
(1) PER A
    PER B
    PER I
    PER J
    I + J = J
    PER J
    FIN
```

II. Hacer el programa para calcular por la fórmula de Simpson

$$\int_a^b \sqrt{X(X^2-4)}\ \text{Arctg}\ (1/X)\ dx$$ Sabiendo que

$$\int_a^b F(x)\,dx = \frac{h}{6}\left(F(x_0) + 4F(x_1) + 2F(x_2) + 4F(x_3) + \ldots + 2F(x_{n-2}) + 4F(x_{n-1}) + F(x_n)\right)$$

Datos: N, $a = x_0$, $b = x_n$

N par $\qquad x_{i+1} = x_i + h \qquad h = \dfrac{b-a}{N}$

```
LEAL
CALCULO DE UNA INTEGRAL POR LA FORMA DE SIMPSON
ENT NI
FLO A B C H S
REF (3)
LEC N
LEC A
LEC B
NNOR = H
B - A/H = H
VAS (1) A, C
0 = I
C = S
(3) A + H = A
```

```
I + 1 = I
VAS (1) A, C
C: 4 + S = S
I − N + 1? SSC (2)
A + H = A
I + 1 = I
VAS (1) A, C
C: 2 + S = S
VEA (3)
(2) A + H = A
VAS (1) A, C
C + S: H/6 = S
PERS
ALT
(1) CRU A
A: A − 4: A RAC = C
1.O/A ACT: C = C
FRU (1) C
FIN
```

III. Hacer el programa para multiplicar dos matrices A_{nm}, B_{mk}. Hagamos la lectura por filas

$$\begin{bmatrix} a(1) & a(2) \ldots\ldots a(m) \\ a(m+1) & a(m+2) \ldots a(2m) \\ \vdots & \vdots \\ a((n-1)m+1) & \ldots\ldots a(n\,m) \end{bmatrix} \begin{bmatrix} b(1) & b(2) \ldots b(k) \\ b(k+1) & b(k+2) \ldots b(2k) \\ \vdots & \vdots \\ b((m-1)k+1) & \ldots b(m\,k) \end{bmatrix} =$$

$$= \begin{bmatrix} c(1) & c(2) \ldots c(k) \\ c(k+1) & c(k+2) \ldots c(2k) \\ \vdots & \vdots \\ c((n-1)k+1) & \ldots c(n\,k) \end{bmatrix}$$

```
LEAL
MULTIPLICACION DE DOS MATRICES
ENT H I J K L M N S
FLO A(250) B(250)
REF (5)
LEC M
LEC N
LEC K
N: M = H
```

```
1 = I
(1) CIC
LEC A(I)
REP (1) I, 1' H
M: K = H
1 = I
(2) CIC
LEC B(1)
REP (2) I, 1' H
1 = I
(3) CIC
1 = H
(4) CIC
0.0 = B
0 = J
(5) CIC
I + J = L
K : J + H = S
A(L) : B(S) + B = B
REP (5) J, 1'M
PER B
REP (4) H, 1' K
REP (3) I, M' K
FIN
```

EJERCICIOS

Hacer los siguientes programas:

I. Hallar por iteración X tal que $X_{n+1} = X_n + a\left(1 - \frac{F(a)}{F(a) - F(X_n)}\right)$

Donde $F(X_n) = \text{Arctg.}\sqrt{\frac{1}{1 - X^2_n}}$ y tal que $|X_{n+1} - X_n| < \varepsilon$

Datos: a, X_0 ε.

II. Dada una lista de N números enteros, hallar la suma de los pares y contarlos, y hallar la suma de los impares y contarlos.

III. Calcular $S = \sum_t \frac{1}{2} f t^2$ para

$f = 3.2,\ 24.7,\ 33,\ -5.3,\ 7.89$

$t = -10,\ -10 + n,\ -10 + 2n\ \ldots,\ 100$ $\quad n$ dato.

IV. Ordenar N números en forma creciente.

V. Calcular un determinante de orden N.

19) **Impresión**

1. *Instrucción DIG*

La instrucción de salida en Leal es PER A, que como se vio en el epígrafe 9 significa imprimir el contenido de la variable A.

En Leal existen dos formas de imprimir el contenido de dicha variable, la formas aritmética y la forma exponencial. Estas formas vienen dadas por la instrucción DIG que puede anteceder o no a la instrucción PER.

a) *DIG m, n*

Esta instrucción indica que las variables flotantes que se vayan a perforar a continuación de esta instrucción, serán impresas en forma aritmética con *m* dígitos antes del punto y con *n* dígitos después del punto. Se imprime: un signo menos si el contenido es negativo o un espacio (sp) si es positivo, los *m* dígitos enteros, el puntos, *n* dígitos fraccionarios y dos espacios finales, o sea $m + n + 4$ caracteres. *m* y *n* son dos enteros mayores o iguales a cero, cualesquiera. Si el contenido de A tiene más de *m* dígitos enteros se imprimen todos, alterándose pues la orden DIG en este caso. Si dicho contenido tiene menos que *m* dígitos enteros, se completan los restantes con espacios, pero si no tiene enteros, pone el cero de las unidades. Si A tiene más que *n* dígitos fraccionarios, se corta la fracción en el enésimo dígito. Y si A tiene menos de *n* dígitos fraccionarios, se completan los restantes con ceros.

Si se va a perforar una variable entera y la última instrucción DIG dada es de la forma DIG *m*, *n* se perforará el entero con m dígitos. Se imprime: un signo menos si el contenido es negativo o un espacio si es positivo, los *m* dígitos enteros y dos espacios finales, o sea, $m + 3$ caracteres. El punto no se imprime.

b) *DIG n*

Esta instrucción indica que las variables que se vayan a perforar a continuación serán impresas en forma exponencial, según potencias de 10, con la mantisa o parte fraccionaria con *n* dígitos y el exponente de 10 correspondiente. Imprime: un signo menos si el contenido es negativo o un espacio si es positivo, el punto, los *n* dígitos de la mantisa, un espacio, una letra *e*, un signo menos si el exponente es negativo o un espacio si es positivo, el exponente de 10 con dos dígitos y dos espacios finales, o sea, $n + 9$ caracteres. *n* puede ser un entero positivo cualquiera.

En la Instrucción DIG *m*, *n*; *m*, *n*, son dos enteros positivos cualesquiera y en DIG *n*; *n* es también un entero positivo cualquiera, ahora si se quiere usar la salida como datos en otro programa, se debe tener en cuenta que sólo se leen enteros hasta 8'388,607 y flotantes con 6 cifras.

EJEMPLO $1 = A$:

```
DIG 6              Perfora          Puede volver a leer
PER A          .100000 e 1

DIG 8                               No puede volver a leer
PER A          .10000000  e 1
```

La instrucción DIG no tiene que ir inmediatamente antes de la instrucción PER. Mientras no aparezca otro DIG se seguirá el formato del último DIG colocado. Si en un programa hay una instrucción PER A antes que cualquier instrucción DIG, se imprime siguiendo el formato de DIG 4, 2.

Las impresiones se hacen una a continuación de la otra en el mismo renglón.

2. *Instrucción LIN*

Se usa cuando se desea imprimir en la próxima línea.

Causa un retorno del carro (cr) y un nuevo renglón (lf). Si se quieren dejar varias líneas, es necesario colocar tantas instrucciones LIN como líneas se quieran dejar.

Según se ha visto anteriormente, las impresiones se hacen una a continuación de la otra en el mismo renglón, pero como el teletipo sólo admite 65 caracteres por renglón, el programador debe tener en cuenta cuando colocar una instrucción LIN que le imprima en la otra línea. De no existir esta instrucción una vez agotados los 65 caracteres correspondiente al renglón, se seguirá imprimiendo sobre el último carácter.

3. *Instrucción ESP*

La instrucción ESP *n*, perfora *n* espacios. Esta instrucción permite encuadrar las impresiones para obtener tablas de resultados.

4. *Instrucción TIT*

Esta instrucción se usa para imprimir algún título o nota explicativa.

Se coloca la palabra TIT y a continuación el título que se quiere imprimir.

Si algunos de los caracteres siguientes:

```
cr      retorno del carro
lf      nuevo renglón
bl      blanco
sp      espacio
```

aparece entre la palabra TIT y el título que se va a imprimir éstos son ignorados. Una vez que se comienza a imprimir el título sólo se ignoran los bl y cr. El final del título se indica por un lf.

La instrucción TIT no da cr, lf, si deseamos darlos antes o después, debe darse una instrucción LIN antes o después, según el caso.

EJEMPLOS DE IMPRESIÓN

```
LEAL
TITULO
ENT NM
FLO  ABCDEFG
REF (1)
.
.
.
.
.
.
.
.
.
.
.
.
.
LIN
TIT RESULTADOS
LIN
PER A                          (antes, no aparece ninguna inst. DIG)
ESP 3
DIG 3, 4
PER B
LIN
LIN
PER C
ESP 2
DIG 3
PER D
LIN
PER N
DIG 2, 2
PER E
LIN
```


BIBLIOTECA
Escuela de Matemát ca

```
PER M
DIG 5
PER F
DIG 2, 0
PER G
FIN
```

Variables	*Valores*
A	2258.1221
B	−3412.20
C	12.25121
D	1.23
E	22.1
F	−219.112
G	17.1
N	−23
M	81

SALE IMPRESO

RESULTADOS

```
 2258.12     −3412.2000

  12.2512       .123 e  1
−.230 e  2    22.10
  81   −.21911 e  3    17.
```

20) **Error**

1. *Error en la compilación*

Un error de sintaxis es detectado durante la compilación señalándose como tal en el programa que se va listando en el telex a la vez que se compila.

Si tenemos un error de compilación, la máquina da cr lf imprimiendo en el siguiente renglón la palabra ERROR y a continuación un número de 4 cifras, que según un código indica el tipo de error cometido, luego da un nuevo cr lf, continuando la compilación en el renglón siguiente en que estaba el error, sólo con el objeto de detectar nuevos posibles errores. Lo que resta de ese renglón será tomado como una observación.

2. *Error en la ejecución del programa objeto.*

Un error en la ejecución del programa objeto (salto a un número de referencia que no aparece en el programa, capacidad excedida, etc.) es detectado imprimindo la palabra ERROR y a continuación un número cuya impresión responderá a la del último DIG dado, este número según un código indica el tipo de error cometido. Y de inmediato se para la ejecución del programa.

21) **Rastreo o Chequeo 2**

Si un programa no se ejecuta correctamente, el rastreo puede usarse para determinar el punto de fallo.

Para el compilador, los renglones de un programa están numerados, comenzando por cero el siguiente a la instrucción REF.

Si se quiere rastrear un renglón, se pone en 1 el interruptor correspondiente al bit más significativo de la consola en el momento en que se vaya a listar dicho renglón durante la compilación, colocándose de nuevo en cero una vez compilado éste. En la ejecución del programa, se imprime el número correspondiente a este renglón, cuando éste se ejecute. Esta impresión responde a la del último DIC dado.

Mediante la facilidad de rastreo se puede determinar el número de veces que en un programa se ejecuta una instrucción. Por ejemplo, si dicho renglón está contenido en un ciclo, se puede determinar la cantidad de veces que se realiza el lazo.

Si se quieren rastrear varios renglones seguidos, basta mantener en 1 el interruptor señalado mientras están compilándose.

El rastreo es realizado por el operador, el programador no tiene que incluir ninguna instrucción en su programa para que éste es realice.

22) **División de un Programa**

Si se tiene un programa que por su longitud no cabe en la memoria de la máquina y el programador es capaz de dividirlo en dos o más partes independientes en la secuencia de operaciones, es posible correr separadas estas partes, conservándose los resultados intermedios. Cada parte ha determinar con la instrucción FIN. La declaración sólo aparece en la primera parte y debe incluir a todas las variables y números de referencias usados en todas las partes.

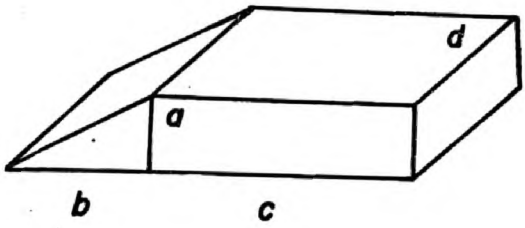


Figura 1

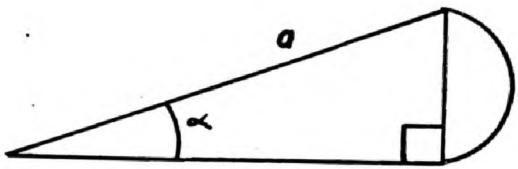


Figura 2

BIBLIOTECA
Escuela de Matemática